1. 最小生成树

```
//不要忘记并查集初始化!!!
#define int long long
int n,m;
struct edge{
    int u,v,w;
}G[1000001];
int fa[1000001];
int find(int x){
    if(fa[x]!=x)fa[x]=find(fa[x]);
    return fa[x];
}
bool cmp(edge _1,edge _2){
    return _1.w<_2.w;
}
int Kruskal(){
    int ans=0,tot=0;
    for(int i=1;i<=m;i++){
        int fx=find(G[i].u);
        int fy=find(G[i].v);
        if(fx!=fy){
            fa[fx]=fy;
            ans+=G[i].w;
            tot++;
            if(tot==n-1){
                return ans;
            }
        }
    }
    return -1;
}
signed main(){
    cin>>n>>m;
    for(int i=1;i<=m;i++){
        cin>>G[i].u>>G[i].v>>G[i].w;
    }
    for(int i=1;i<=n;i++){
        fa[i]=i;
    }
    sort(G+1,G+m+1,cmp);
    int tmp=Kruskal();
    if(tmp==-1)cout<<"orz"<<endl;
    else cout<<tmp<<endl;
```

```
    return 0;
}
```

2. Dijkstra

```
struct edge{
    int v,w;
};
struct node{
    int x,d;
    friend bool operator < (node tmp_x,node tmp_y){
        return tmp_x.d>tmp_y.d;
    }
};
vector<edge>e[100001];
int dis[100001],vis[100001];
priority_queue<node>q;
void dijkstra(int n,int s){
    memset(dis,0x3f,sizeof(dis));
    while(!q.empty())q.pop();
    q.push({s,0});
    dis[s]=0;
    while(!q.empty()){
        int now=q.top().x;
        q.pop();
        if(!vis[now]){
            vis[now]=1;
            for(auto ed:e[now]){
                int v=ed.v;
                int w=ed.w;
                if(dis[v]>dis[now]+w){
                    dis[v]=dis[now]+w;
                    q.push({v,dis[v]});
                }
            }
        }
    }
}
```

3. SPFA（判负环）

```
struct edge{
    int v,w;
};
```

```
struct node{
    int x,d;
};
vector<edge>G[200005];
queue<int>q;
int dis[200005];
int vis[200005];
int tot[200005];
bool spfa(){
    memset(dis,0x3f,sizeof(dis));
    memset(tot,0,sizeof(tot));
    while(!q.empty())q.pop();
    q.push(1);
    dis[1]=0;
    vis[1]=1;
    while(!q.empty()){
        int u=q.front();
        q.pop();
        vis[u]=0;
        for(auto i:G[u]){
            if(dis[i.v]>dis[u]+i.w){
                dis[i.v]=dis[u]+i.w;
                tot[i.v]=tot[u]+1;
                if(tot[i.v]>=n){
                    return true;
                }
                q.push(i.v);
                vis[i.v]=1;
            }
        }
    }
    return false;
}
```

4. 拓扑排序

```
#include<bits/stdc++.h>
using namespace std;
int n;
int deg[1001];
//deg[] : 入度
vector<int>G[1001];
queue<int>q;
int main(){
    cin>>n;
```

```
    for(int i=1;i<=n;i++){
        int x;
        while(cin>>x&&x){
            G[i].push_back(x);
            deg[x]++;
        }
    }
    for(int i=1;i<=n;i++){
        if(deg[i]==0){
            q.push(i);
        }
    }
    while(!q.empty()){
        int u=q.front();
        cout<<u<<" ";
        q.pop();
        for(auto v:G[u]){
            deg[v]--;
            if(deg[v]==0){
                q.push(v);
            }
        }
    }
    return 0;
}
```